CO1-AT1

CSA4316-Internet Programming

Name: K.Naga Venkata Siva
Reg.no: 192425345

Web Technology: HTML5, HTTP and Form Handling

1. Online Symposium Portal

A. Identify appropriate HTML5 semantic elements missing

The given HTML form lacks a complete HTML5 document structure and several semantic and accessibility features.

- <header> – identifies the introductory content or page heading.
- <main> – represents the primary content of the page.
- <section> – groups related content, such as symposium registration.
- <footer> – contains copyright or contact information.
- <label> – provides accessible labels for input fields.
- <fieldset> and <legend> – group related form controls and describe the group.

B. Modified standards-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
<body>
  <header>
    <h1>Online Symposium Portal</h1>
  </header>

  <main>
    <section aria-labelledby="registration-heading">
      <h2 id="registration-heading">
        Student Registration
```

```
        </h2>

        <form action="/register" method="POST">
          <fieldset>
            <legend>Personal Information</legend>

            <label for="name">Full Name:</label>
            <input type="text" id="name" name="name"
                autocomplete="name" required>

            <label for="email">Email Address:</label>
            <input type="email" id="email" name="email"
                autocomplete="email" required>

            <button type="submit">Register</button>
          </fieldset>
        </form>
      </section>
    </main>

    <footer>
      <p>&copy; 2026 Online Symposium Portal</p>
    </footer>
</body>
</html>
```

C. Interpret the HTTP request generated during form submission

When the student submits the form, the browser sends an HTTP POST request to the server.

Example request:

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=Arun&email=arun%40example.com
```

Explanation:

- POST: Sends form data in the request body.
- /register: The destination endpoint on the server.
- Content-Type: Specifies the format of the submitted data.
- Request body: Contains the values entered by the student.
- HTTP version: Indicates the protocol version used in this illustrative request.

The actual HTTP version and headers depend on the browser and network connection.

D. Analyze the HTTP response cycle



E. Improvements for usability and accessibility

1. Associate every input with a visible <label>.
2. Use required and appropriate input types for client-side validation.
3. Provide clear error messages and success notifications.
4. Use keyboard-accessible controls and visible focus indicators.
5. Maintain sufficient color contrast and readable font sizes.
6. Use HTTPS to protect information in transit.
7. Validate data on the server even when browser validation is enabled.
8. Prevent duplicate registrations and provide clear submission feedback.

2. Cross-Browser Compatibility Issue

A. Identify improper nesting and missing closing tags

Given code:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

Issues:

- The `<h1>` heading is missing its explicit closing `</h1>` tag.
- The `<p>` paragraph is missing its explicit closing `</p>` tag.
- The document is missing `<!DOCTYPE html>`.
- The `<html>` element lacks a language attribute.
- The document is missing a character encoding declaration and viewport metadata.
- The content uses a generic `<div>` instead of suitable semantic elements.

B. How different browsers may interpret the structure

Browsers use HTML parsing rules to construct the Document Object Model (DOM).

- Automatic error recovery: Browsers generally insert implied closing tags where HTML rules permit.
- DOM differences: Invalid or ambiguous nesting can result in an unexpected DOM structure.
- Rendering differences: Default heading margins, paragraph spacing, fonts and other styles can vary across browsers.
- Maintenance problems: CSS selectors and JavaScript may behave unexpectedly if the resulting DOM differs from what the developer intended.

Modern browsers generally recover from this example similarly, but malformed markup should not be relied upon.

C. Missing semantic elements

- <header> – contains the company heading or introductory content.
- <main> – identifies the primary page content.
- <section> – groups the company's introduction or related information.
- <footer> – contains copyright and contact information when needed.

Semantic elements improve document structure, accessibility and maintainability. They do not automatically guarantee cross-browser consistency.

D. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>Company</title>
</head>
<body>
   <header>
      <h1>Welcome</h1>
   </header>

   <main>
      <section aria-labelledby="about-heading">
         <h2 id="about-heading">About Us</h2>
         <p>
            Learn more about our company,
            our mission and our services.
         </p>
      </section>
   </main>

   <footer>
      <p>&copy; 2026 Company</p>
   </footer>
</body>
</html>
```

E. Techniques to ensure cross-browser compatibility

1. Validate HTML: Use the W3C Markup Validation Service to identify markup errors. Validate HTML .
2. Use CSS resets or normalization: Reduce inconsistent browser default styling.
3. Apply responsive design: Use flexible layouts, relative units and media queries.
4. Test across browsers: Check current versions of Chrome, Firefox, Safari and Edge.
5. Use feature detection: Check whether a browser supports a feature before relying on it.
6. Avoid deprecated elements: Follow modern HTML and CSS standards.
7. Test accessibility: Verify keyboard navigation, screen-reader semantics and color contrast.
8. Use automated testing: Run cross-browser tests using tools such as Playwright.

3. Form Submission Failure

A. Identify issues in the form configuration

Given code:

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

The main issues are:

- Incorrect button type: type="button" creates a regular button and does not submit the form automatically.
- GET method for a password: GET places submitted form values in the URL query string, potentially exposing the password in browser history, server logs and other URL-handling systems.
- Missing labels: The fields lack explicit accessible labels.
- Missing validation: The username and password are not marked as required.
- No HTTPS guarantee: The markup alone does not ensure encrypted communication.

B. Why is the HTTP request not triggered?

The browser does not automatically submit a form when a user clicks a button with type="button".

This button is intended for custom JavaScript actions, such as opening a dialog or running a script. Since no JavaScript submission handler is provided, clicking it does not trigger the normal form submission process.

Changing the button to type="submit" allows the browser to submit the form using the configured method and destination.

C. Interpret the role of the GET method

The GET method requests a resource and conventionally places form values in the URL query string.

For example:
GET /submit?username=arun&password=secret123 HTTP/1.1
Host: example.com

This is an illustrative request showing the potential exposure; the original form does not generate this request merely by clicking its type="button" button.

Limitations of GET for login forms:

- Credentials can appear in browser history and server or proxy logs.
- URLs may be copied, bookmarked or shared.
- URLs can be transmitted in referrer information in some circumstances.
- GET requests are intended primarily for retrieval, not for operations that change server state.

D. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>Login</title>
</head>
<body>
   <main>
```

```
    <h1>Login</h1>

    <form action="/submit" method="POST">
      <div>
        <label for="username">Username:</label>
        <input type="text" id="username"
            name="username"
            autocomplete="username"
            required>
      </div>

      <div>
        <label for="password">Password:</label>
        <input type="password" id="password"
            name="password"
            autocomplete="current-password"
            required>
      </div>

      <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

E. Improvements for secure data transmission

- Use HTTPS everywhere: Encrypt credentials and other sensitive data in transit.
- Use POST for login submissions: Keep credentials out of the URL query string.
- Hash passwords securely on the server: Use established password-hashing algorithms such as Argon2id, scrypt or bcrypt with appropriate parameters.
- Validate input on the server: Never rely exclusively on client-side validation.
- Protect sessions: Use secure session cookies with HttpOnly, Secure and appropriate SameSite settings.
- Apply login rate limits: Reduce the risk of brute-force attacks.
- Avoid logging passwords: Never write raw credentials to application logs.
- Return safe error messages: Avoid revealing whether a particular username exists.
- Use CSRF protection where appropriate: Protect state-changing requests when the application's authentication model requires it.